CONFIDENTIAL · SECURITY AUDIT

VERSION 7 · RE-REVIEW

Security Audit
Tech for Palestine
Website

techforpalestine.org · Astro 5 SSR on Cloudflare Pages

This is the seventh-round re-review, and the first with a substantial gap behind it. It covers **115 commits / 114 merged PRs** landed between v6 (2026-04-14) and 2026-09-22 — 296 files, +43,384 / −5,963 lines, five new API routes and two removed. All 25 findings closed in rounds v1–v6 were re-verified and **all 25 remain fixed**. **Ten new findings** were identified, including one high-severity issue: the signature-verified QGiv payment webhook was replaced by a client-triggered endpoint, so a privileged Hub membership invite can now be obtained by anyone who can set an HTTP header. Six of the ten are in code added since v6; four are in code that predates it and had not been examined in earlier rounds.

V1 ORIGINAL	V2 RE-REVIEW	V3 RE-REVIEW	V4 RE-REVIEW	V5 RE-REVIEW	V6 RE-REVIEW	V7 RE-REVIEW
2026-03-31	2026-04-02	2026-04-06	2026-04-09	2026-04-13	2026-04-14	**2026-09-22**

Executive Summary — v7 Re-review

Five months and 115 commits after v6. Ten new findings; every previously closed finding still holds.

25	0	1	4	6	11
TOTAL FIXED	CRITICAL	HIGH	MEDIUM	LOW	OPEN

Remediation progress: 25 of 36 findings resolved (69%)

69%

⚠ Priority — Payment Verification Was Removed From the Membership Flow

In PR #489 the signature-verified QGiv webhook (`qgiv-webhook.ts`) and the Bearer-authenticated `membership-invite.ts` were deleted and replaced with `/api/membership-complete`, which the browser calls from a QGiv `donationComplete` DOM event. Its only access control is an `Origin` header check. **Confirmed live on production:** a plain `curl` that sets `Origin: https://techforpalestine.org` passes the gate and reaches the business logic, so a paid-tier Hub invite and an EmailOctopus subscription can be triggered for any address without a payment ever taking place. This is the same class of issue as C-2/C-3, which were closed in v2. See **S-1**.

◖ Rate Limiting Holds, But The Uncached Endpoints Fail Before It Engages

Both WAF rules confirmed live. POST throttles early (429 after roughly ten requests in a 30-request burst). GET throttles far later than the 15 requests / 60 seconds v6 recorded — a 40-request burst on `/api/faq` returned 40 × 200, and only a 300-request burst reached the limit, at around the 77th request. In that same run thirteen responses came back **500 before or alongside the first 429s**: `/api/faq` queries Notion once per request with no cache, so sustained load reaches Notion and the endpoint breaks before the edge protects it. Caching those reads matters more here than the threshold does. See **S-4**.

✓ What Held Up

The v1–v6 remediations have survived a near-total rewrite of the front end. No secrets are committed anywhere in the tree; the nonce-based CSP still carries no `'unsafe-inline'` on either `script-src` or `style-src`; the `project-proxy` path normalisation and header allowlist are intact; all `/api/*` responses are `no-store`; form endpoints still validate and origin-gate; and the two new secret comparisons added since v6 (HTTP Basic admin auth and the Sentry webhook HMAC) both use `constantTimeEqual()` rather than `===`.

SCOPE OF THIS REVIEW

AREA	WHAT CHANGED SINCE V6, AND WHAT WAS EXAMINED
New API routes	`membership-complete` , `donation-complete` , `pipe` , `open-roles` , `sentry-webhook` , `admin/conversion-stats`
Removed API routes	`qgiv-webhook` , `membership-invite` (both deleted in #489)
New auth surface	HTTP Basic admin area (`/admin/conversions`), Sentry→Mattermost webhook relay, shared origin policy helper
New data paths	ICS events feed (token in URL), ProjectHub Workers-cache layer, Hub open-roles, Cloudflare KV conversion log
New third parties	Sentry (client + server), GrowthBook, first-party-proxied Plausible script, CharityStack, ValidAid
Re-examined	Areas unchanged since v6 but not previously covered in depth: response headers on SSR routes, the nonce `meta` mechanism, dependency advisories

COMPLETE ISSUE REGISTER — V7 STATUS (1 OF 2)

ID	FINDING	PEAK SEVERITY	V7 STATUS
C-1	API key & account ID in wrangler configs	CRITICAL	Fixed v2 · re-verified
C-2	Unauthenticated membership invite endpoint	CRITICAL	Fixed v2 · route deleted
C-3	Unverified webhook triggers privileged invite	CRITICAL	Fixed v2 · route deleted
N-1	Webhook secret exposed in URL query param	CRITICAL	Fixed v3 · re-verified
N-2	Timing attack on token comparison	HIGH	Fixed v3 · re-verified
H-1	Secret key logged to browser console	HIGH	Fixed v3 · re-verified
H-2	Full donor PII payload echoed in HTTP response	HIGH	Fixed v4 · re-verified
H-3	Middleware caches POST/API responses as public	HIGH	Fixed v3 · re-verified
H-4	No rate limiting on any endpoint	HIGH	Fixed v6 · threshold: S-4
H-5	Wildcard CORS on all endpoints	HIGH	Fixed v3 · re-verified
N-3	membership-invite proxies arbitrary body fields	MEDIUM	Fixed v3 · route deleted
M-1	No CSRF / Origin check on form endpoints	MEDIUM	Fixed v4 · re-verified
M-2	No input format/length validation on forms	MEDIUM	Fixed v4 · re-verified
M-3	getEnv globalThis fallback	MEDIUM	Fixed v3 · re-verified
M-4	Donor PII (email) in server logs	MEDIUM	Fixed v4 · re-verified
M-5	Internal error.message in HTTP responses	MEDIUM	Fixed v3 · re-verified
M-6	Dual middleware conflict — cache-control dead code	MEDIUM	Fixed v6 · re-verified
R-1	PUBLIC_SECRET_KEY in client-side Axios bundle	MEDIUM	Fixed v4 · re-verified
R-2	CSP 'unsafe-inline' defeats XSS protection	MEDIUM	Fixed v4 · re-verified
P-1	project-proxy: Authorization forwarded to any path	MEDIUM	Fixed v5 · re-verified
P-2	project-proxy: dot-segment traversal bypass	MEDIUM	Fixed v6 · re-verified
L-1	Notion DB IDs in wrangler.toml	LOW	Fixed v3 · re-verified
L-3	Image proxy has no authentication	LOW	Fixed v6 · worker gone
L-4	style-src still uses 'unsafe-inline' in CSP	LOW	Fixed v6 · re-verified
L-5	project-proxy forwards all browser headers upstream	LOW	Fixed v6 · re-verified
L-2	QGiv embed script loaded without Subresource Integrity	LOW	Open since v1

COMPLETE ISSUE REGISTER — NEW IN V7 (2 OF 2)

ID	FINDING	SEVERITY	STATUS
S-1	Membership/donation completion endpoints grant Hub invites and list writes behind a spoofable Origin check	HIGH	Open · new v7
S-2	Security response headers absent on every SSR page — `_headers` now only reaches static assets	MEDIUM	Open · new v7
S-3	Authenticated admin dashboard served with `Cache-Control: public, max-age=600`	MEDIUM	Open · new v7
S-4	Uncached Notion-backed GET routes return 500 under load before the GET rate limit engages	LOW	Open · new v7
S-5	Token-bearing events feed URL reaches error telemetry unscrubbed	MEDIUM	Open · new v7
S-6	Astro runtime two majors behind, with published RCE/SSRF/XSS advisories in the pinned range	MEDIUM	Open · new v7
S-7	Per-request CSP nonce republished in a DOM-readable `<meta>` tag	LOW	Open · new v7
S-8	Raw-HTML sink renders upstream error text in the volunteer form	LOW	Open · latent
S-9	`/api/pipe` accepts requests with no Origin and from any `*.pages.dev` host	LOW	Open · new v7
S-10	Third-party analytics script re-served first-party without integrity pinning	LOW	Open · new v7

LIVE VERIFICATION LOG — 2026-09-22, AGAINST PRODUCTION

PROBE	RESULT	INTERPRETATION
`GET /`	200, CSP present; no XFO / nosniff / Referrer-Policy	S-2 confirmed
`GET /favicon.png`	All four headers present	`_headers` applies to assets only
`GET /admin/conversions`	401 + `Cache-Control: public, max-age=600`	S-3 confirmed
`GET /api/events`	200, `no-store`, `ACAO: *`	H-3 / H-5 still correct
`POST /api/membership-complete` (no Origin)	403	Origin gate active
`POST /api/membership-complete` (forged Origin, invalid email)	400 "Invalid or missing email"	S-1 confirmed — gate passed
40 × concurrent `GET /api/faq`	40 × 200, no 429	Below the GET threshold
300 × concurrent `GET /api/faq`	~76 × 200, then 13 × 500 mixed with 429s	S-4 — GET rule active but late; upstream fails first
30 × concurrent `POST /api/donation-complete`	Mixed 400 / 429	POST rate limit active
`GET /api/project-proxy?path=/api/method/../../api/auth/admin`	503 "Proxy not configured"	Proxy unconfigured in prod; guard verified in code

No test wrote data. The two completion-endpoint probes used a deliberately invalid email address so the request was rejected at validation, after the access-control decision but before any Hub or EmailOctopus call.

HIGH S-1 **Membership and Donation Completion Endpoints Grant Hub Invites and Mailing-List Writes Behind a Spoofable Origin Check**

Files: src/pages/api/membership-complete.ts:37 · src/pages/api/donation-complete.ts:22 · src/components/membership/QgivJoin.tsx:181–212 | **OWASP:** A01:2021 – Broken Access Control | **Introduced:** PR #489

Until PR #489 the membership pipeline was server-verified: QGiv called `/api/qgiv-webhook` with an `X-Webhook-Secret` header, the handler compared it with `constantTimeEqual()`, and only then called the Bearer-authenticated `/api/membership-invite`. Both routes were deleted. The replacement runs in the browser: a QGiv `donationComplete` DOM event fires in `QgivJoin.tsx` and the island POSTs the contact details to `/api/membership-complete`.

That endpoint performs two privileged actions — a Hub invite of `type: "paid"` and an EmailOctopus subscription tagged `member` — and its only access control is an `Origin` header comparison. `Origin` is a browser-enforced header: it constrains other web pages, and nothing else. Any HTTP client sets it at will.

```
# Confirmed against production, 2026-09-22
$ curl -X POST https://techforpalestine.org/api/membership-complete \
    -H 'Content-Type: application/json' \
    --data '{"email":"not-an-email"}'
403   # no Origin header — rejected

$ curl -X POST https://techforpalestine.org/api/membership-complete \
    -H 'Content-Type: application/json' \
    -H 'Origin: https://techforpalestine.org' \
    --data '{"email":"not-an-email"}'
400 {"message":"Invalid or missing email"}   # gate passed; only the email format stopped it
```

The 400 is the validator, not the guard — the request is already inside the handler. Substituting a well-formed address completes the flow. There is no transaction id, no QGiv lookup, and no signature anywhere in the path, so the server has no way to distinguish a real payment from a forged call.

Impact. Anyone can mint a paid-tier Hub invite for an address they control, which is unpaid access to the member chat and teams. Anyone can also subscribe and tag an arbitrary third-party address to the members or donor list, which is both a mail-bombing primitive aimed at a victim and a way to corrupt the membership data that drives reporting. `/api/donation-complete` is the same shape with the EmailOctopus half only. The POST rate limit caps the rate, not the act.

FIX — PUT PAYMENT VERIFICATION BACK ON THE SERVER

```
// Preferred: restore the webhook. QGiv posts the transaction server-to-server;
// the invite decision is then made from data the browser never touched.
// src/pages/api/qgiv-webhook.ts
const secret = getEnv("QGIV_WEBHOOK_SECRET", locals);
const sent = request.headers.get("X-Webhook-Secret") ?? "";
if (!secret || !constantTimeEqual(sent, secret)) return new Response("Unauthorized", { status: 401 });

// Minimum, if the client-side trigger has to stay: prove the transaction exists
// before acting on it, and never accept the tier or the email from the caller.
const txn = await fetch(`${QGIV_API}/transactions/${transactionId}`, { headers: { Authorization: qgivKey } });
if (!txn.ok) return new Response(JSON.stringify({ message: "Unknown transaction" }), { status: 402 });
const { email, tier } = await txn.json();   // authoritative values, from QGiv
```

Keep the `Origin` check — it is useful defence in depth against drive-by cross-site calls — but it cannot be the only control on an endpoint that grants access or writes to a mailing list.

MEDIUM S-2 **Security Response Headers Are Absent On Every Server-Rendered Page**

Files: public/_headers · src/middleware/csp.ts:23–34 · astro.config.mjs:18 | **OWASP:** A05:2021 – Security Misconfiguration

`public/_headers` sets `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy` and `Permissions-Policy` on `/*`. On Cloudflare Pages that file is applied by the static asset handler. The site runs `output: "server"`, so nearly every HTML route is produced by a Pages Function instead, and those responses never pass through the asset handler — the rules reach the favicon and miss the site. This predates v6; it is reported here because earlier rounds examined the CSP without checking the other four headers on a live SSR response.

```
# 2026-09-22
$ curl -sI https://techforpalestine.org/favicon.png
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(), microphone=(), geolocation=()

$ curl -sD- -o/dev/null https://techforpalestine.org/
content-type: text/html
cache-control: public, max-age=600
content-security-policy: default-src 'self'; script-src 'nonce-…'; …
# none of the four headers above
```

The CSP has no `frame-ancestors` directive either, so with `X-Frame-Options` gone there is nothing stopping any site from framing techforpalestine.org. The donate and membership pages carry live payment widgets, which makes UI-redress the concrete risk rather than a theoretical one.

FIX — SET THEM IN MIDDLEWARE, WHERE SSR RESPONSES ACTUALLY PASS

```
// src/middleware/cache-control.ts (or a new securityHeaders middleware in the sequence)
response.headers.set("X-Content-Type-Options", "nosniff");
response.headers.set("Referrer-Policy", "strict-origin-when-cross-origin");
response.headers.set("Permissions-Policy", "camera=(), microphone=(), geolocation=()");

// and in src/middleware/csp.ts — the modern equivalent of X-Frame-Options
"frame-ancestors 'none'",
```

Keep `public/_headers` as it is: it still covers the static assets, and the two do not conflict.

MEDIUM S-3 **The Authenticated Admin Dashboard Is Served With `Cache-Control: public, max-age=600`**

Files: src/middleware/cache-control.ts:9–11 · src/pages/admin/conversions.astro | **OWASP:** A05:2021 – Security Misconfiguration

The cache middleware marks any GET response that is not under `/api/` and does not already carry a `Cache-Control` header as `public, max-age=600`. The HTTP Basic-protected conversion dashboard added since v6 matches that rule.

```
$ curl -sD- -o/dev/null https://techforpalestine.org/admin/conversions
HTTP/2 401
www-authenticate: Basic realm="Admin"
cache-control: public, max-age=600
```

Per RFC 9111 §3.5 a shared cache must not store a response to a request carrying `Authorization` — unless the response says `public`, which is the one directive that lifts the prohibition. The header is therefore explicit permission for any intermediary to keep the dashboard and hand it to the next person who asks for that URL. Cloudflare's edge does not cache Function HTML by default, which limits this in the current deployment, but the browser disk cache and any corporate proxy in front of an admin both apply today.

FIX — NEVER MARK AN AUTHENTICATED ROUTE PUBLIC

```
// src/middleware/cache-control.ts
const isApi = pathname.startsWith("/api/");
const isPrivate = pathname.startsWith("/admin/");
if (!response.headers.has("Cache-Control") || isApi || isPrivate || !isGet) {
  response.headers.set("Cache-Control", (!isGet || isApi || isPrivate) ? "no-store" : "public, max-age=600");
}
```

Worth considering separately: `public` is a strong default for every other SSR page too. `/donate` varies its content on the visitor's country (`isUK`), and a ten-minute public entry does not distinguish between them.

LOW S-4 **Uncached Notion-Backed GET Routes Fail Under Load Before The GET Rate Limit Engages**

Files: src/pages/api/faq.ts · ideas.ts · speakers.ts · e4p-signatories.ts · events.ts · Cloudflare WAF configuration | **OWASP:** A04:2021 – Insecure Design

v6 recorded a GET limit of 15 requests / 60 seconds / IP. A 40-request burst on 2026-09-22 returned 40 × 200, which on its own reads as the rule having been removed. A 300-request burst shows otherwise: the rule is in place but set far higher, with the first 429 arriving around the 77th request and 429s mixed with 200s for the rest of the run. The more interesting part of that run is what came back in between:

```
# 300 × concurrent GET /api/faq — excerpt around the transition
… 200 200 200 500 200 200 500 429 429 500 500 500 500 500 500 500 500 500 429 200 200 200 429 429 …
```

Thirteen 500s, several of them *before* the first 429. `/api/faq` issues a Notion API query per request with no cache, so sustained load reaches Notion and the endpoint fails before the edge throttles it. `/api/ideas`, `/api/speakers`, `/api/e4p-signatories` and `/api/events` have the same shape. `/api/projects` is the exception and shows the right pattern — `projectsClient.ts` keeps a Workers Cache entry, fresh for five minutes and usable stale for 24 hours.

Two things follow. The protection is real, so this is not the regression a smaller probe suggested, and it is rated low. But the threshold sits above the point where the uncached endpoints break, which means a single client can still push those routes into upstream failure — and the ceiling is well above the 15/60 figure recorded at v6, so whatever the rule is today, it is not what the last report documented.

FIX — CACHE THE UPSTREAMS FIRST, THEN SET THE THRESHOLD DELIBERATELY

- Wrap the Notion and ICS reads in the same `resolveWithCache()` helper `projectsClient.ts` already uses. It removes the failure mode entirely and makes the pages faster and outage-tolerant at the same time.
- Re-derive the intended GET threshold and record it alongside the rule; it should sit below the point at which the upstreams start returning 500, not above it.
- Check whether those thirteen 500s reached Sentry. If a burst from one IP can fail an endpoint without raising an alert, the monitoring gap is worth more attention than the rate limit.

MEDIUM S-5 **The Token-Bearing Events Feed URL Reaches Error Telemetry Unscrubbed**

Files: src/store/eventsClient.ts:373–393 · src/lib/report-error.ts · sentry.server.config.js | **OWASP:** A09:2021 – Security Logging & Monitoring Failures

`.env.example` documents `EVENTS_ICS_URL` as a feed URL that "includes an auth token as a query param". `fetchEvents()` passes that URL straight to `fetch()`, and every failure goes to `reportError()`, which does `console.error(error)` and `Sentry.captureException(error)`.

```
// src/store/eventsClient.ts
const response = await fetch(icsUrl);          // URL carries ?token=…
…
} catch (error) {
  reportError(error, { context: "events-ics-fetch" });   // console.error + Sentry
}
```

Two paths carry the secret out. A rejected `fetch` can put the requested URL into the error text, which lands in the Workers log stream. More reliably, the Sentry Cloudflare SDK instruments outbound `fetch` and attaches the request URL as a breadcrumb or span to whatever error is captured next — and neither Sentry config sets `beforeSend` or `beforeBreadcrumb`, so nothing strips the query string. Anyone with read access to the Sentry project would then hold a working credential for the calendar feed.

FIX — SCRUB THE QUERY STRING, THEN ROTATE

```
// sentry.server.config.js
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  beforeBreadcrumb(crumb) {
    const url = crumb.data?.url;
    if (typeof url === "string" && url.includes("?")) crumb.data.url = url.split("?")[0] + "?[redacted]";
    return crumb;
  },
});
```

- Check the existing Sentry issues for the feed host before assuming this has not already happened; if the token appears, rotate it at the calendar source.
- If the Mattermost calendar plugin can authenticate by header instead of query param, that removes the problem at the root rather than filtering it downstream.

MEDIUM S-6 **The Astro Runtime Is Two Majors Behind, With Published RCE, SSRF And XSS Advisories In The Pinned Range**

Files: package.json · package-lock.json (astro 5.18.1) | **OWASP:** A06:2021 – Vulnerable & Outdated Components

`npm audit` reports 42 advisories across the tree (2 critical, 20 high). Most sit in build-time or Node-only packages — `tar`, `browserslist`, `postcss`, `undici`, `vite` — which never reach the Workers runtime and are not urgent. The framework itself is the part that does.

PACKAGE	PINNED	ADVISORY AFFECTING THAT VERSION	FIXED IN
astro	5.18.1	RCE through AVIF image optimization (GHSA-26w7-cxv4-gfx2, CVSS 9.8)	7.2.8
astro	5.18.1	Host-header SSRF in prerendered error page fetch (GHSA-2pvr-wf23-7pc7, 7.5)	6.4.6
astro	5.18.1	Reflected XSS via unescaped slot name (GHSA-8hv8-536x-4wqp, 7.1)	6.3.3
astro	5.18.1	XSS in `define:vars` via incomplete `</script>` sanitization (GHSA-j687-52p2-xcff, 6.1)	6.1.6
svelte	5.55.5	SSR XSS via spread attributes; XSS via DOM clobbering (GHSA-pr6f-5x2q-rwfp, GHSA-rcqx-6q8c-2c42)	5.55.7
axios	1.16.0	Prototype-pollution gadgets in request construction (GHSA-mmx7-hfxf-jppx and others)	1.18.0

Two mitigations apply and should be weighed before this is treated as an emergency. The AVIF RCE is in Astro's sharp-backed image service; this project configures `imageService: "cloudflare"`, so that code path is not the one in use. The `define:vars` XSS needs attacker-controlled data inside the directive, and the two uses in the tree pass a boolean derived from Cloudflare's geo header. The slot-name and Host-header issues have no such qualifier.

Two dependencies are also declared but never imported anywhere in `src/`: `react-router-dom@7` (which carries its own CSRF and open-redirect advisories) and `install@0.13.0`. They add advisory noise and install surface for nothing.

FIX — PATCH THE RUNTIME, PRUNE THE REST

- Bump `svelte` to ≥ 5.55.7 and `axios` to ≥ 1.18.0 — both are in-range, low-risk updates.
- Plan the Astro 5 → 7 upgrade as its own piece of work with a preview deployment; it is two majors and will need the adapter and integrations moved in step.
- `npm uninstall react-router-dom install`.
- Add `npm audit --omit=dev --audit-level=high` to CI so the next runtime-reachable advisory shows up at the PR rather than at the next audit.

LOW S-7 **The Per-Request CSP Nonce Is Republished In A DOM-Readable `<meta>` Tag**

Files: src/layouts/Layout.astro:58,81 · HomeLayout.astro:97 · AdminLayout.astro:36 | **OWASP:** A05:2021 – Security Misconfiguration

Each layout emits `<meta name="csp-nonce" content={Astro.locals.cspNonce}>` so an inline bootstrap can copy the nonce onto `<style>` elements that MUI and Emotion create at runtime. CSP deliberately hides the `nonce` content attribute from DOM reads for exactly this reason: a nonce that scripts and markup can read is a nonce that can be lifted and reused. Publishing it in a `meta` tag puts it back.

The practical exposure today is limited — reading the tag needs script execution, and a script that already runs has no use for the nonce. It is a weakening of the guarantee rather than a live hole, which is why it is rated low. It becomes material if a markup-injection sink ever appears above the `meta` tag in the document, or if one of the allowed third-party origins is used to read it.

The same workaround also patches three globals on every page — `document.createElement`, `Element.prototype.setAttribute` and `Element.prototype.innerHTML` — the last of which rewrites markup with a regex before the parser sees it. That is a lot of load-bearing machinery to keep a strict `style-src`, and it is worth revisiting whether Emotion's own nonce option, wired at build time, can replace it.

FIX — HAND THE NONCE TO EMOTION DIRECTLY INSTEAD OF PUBLISHING IT

```
// Emotion accepts a nonce on its cache; no meta tag and no prototype patching needed.
createCache({ key: "mui", nonce: cspNonce });
```

If the meta tag has to stay for now, at minimum keep it off pages that render third-party or user-derived HTML, and re-audit after each new embed is added.

LOW S-8 **Raw-HTML Sink Renders Upstream Error Text In The Volunteer Form**

File: src/components/volunteerForm.tsx:157 (and :100) | **OWASP:** A03:2021 – Injection | **Status:** latent — the proxy it depends on is unconfigured in production

```
<div key={index} dangerouslySetInnerHTML={{ __html: `Error: ${err.message}` }} />
```

`err.message` comes from the ProjectHub/Frappe application API by way of `/api/project-proxy`. Frappe validation errors routinely quote the submitted value back, so a crafted field can return as live markup. The nonce CSP blocks script execution and inline event handlers, which leaves markup injection — an overlaid link or a fake input inside a form the user already trusts.

It is not currently reachable: `PUBLIC_API_URL` and `PUBLIC_SECRET_KEY` are not set in production, so the proxy answers 503 "Proxy not configured" and the form never renders an upstream error. That makes it a latent issue to fix before the proxy is switched back on, not an incident. The same component also logs the full set of applicant form values to the browser console at line 100.

FIX — RENDER IT AS TEXT, DROP THE CONSOLE LOG

```
<div key={index}>{`Error: ${err.message}`}</div>
```

LOW S-9 `/api/pipe` Accepts Requests With No Origin And From Any `*.pages.dev` Host

File: src/pages/api/pipe.ts:8–11, 65–88 | **OWASP:** A01:2021 – Broken Access Control

```
const ORIGIN_POLICY: OriginPolicy = {
  allowedSuffixes: [".pages.dev"],      // any Cloudflare Pages site, not just ours
  allowMissingOrigin: true,          // any non-browser client
};
```

The analytics relay is the one endpoint that accepts a missing `Origin`, which means any server-side client can post events. Two consequences: Plausible conversion data can be polluted by anyone who finds the route, and when Plausible drops an event the handler writes a record built from the caller's own body into the `DROPPED_CONVERSIONS` KV namespace — attacker-controlled data in the store that backs the admin dashboard. The POST rate limit bounds the volume.

The `.pages.dev` suffix is also broader than intended: anyone can deploy a Cloudflare Pages project. The other two endpoints get this right by pinning `.website-aun.pages.dev`.

FIX — MATCH THE POLICY THE OTHER ENDPOINTS ALREADY USE

```
const ORIGIN_POLICY: OriginPolicy = { allowedSuffixes: [".website-aun.pages.dev"] };
// allowMissingOrigin dropped — the browser always sends Origin on a cross-origin POST,
// and the beacon this relays is always sent from a page.
```

LOW S-10 Third-Party Analytics Script Is Re-Served First-Party Without Integrity Pinning

File: src/pages/js/web.js.ts | **OWASP:** A08:2021 – Software & Data Integrity Failures

The route fetches `plausible.io/js/pa-….js` server-side and re-serves the body from the site's own origin with `public, max-age=86400`. The script then runs as same-origin code under `script-src 'self'` — outside the CSP allowlist that would otherwise scope it, with no integrity check, and cached for a day. A bad response from upstream is therefore first-party JavaScript on every page for up to 24 hours. Same class as L-2, different vendor.

FIX — PIN THE EXPECTED HASH AND FAIL CLOSED

```
const body = await response.arrayBuffer();
const digest = await crypto.subtle.digest("SHA-384", body);
if (toBase64(digest) !== EXPECTED_SHA384) {
  reportError(new Error("Plausible script hash mismatch"), { context: "web.js" });
  return new Response("", { status: 502 });
}
```

As with L-2 this pins a version: agree an update path with the vendor, or shorten the cache TTL so a mismatch is caught and alerted on quickly.

STILL OPEN FROM EARLIER ROUNDS

LOW L-2 QGiv Payment Embed Script Loaded Without Subresource Integrity — Open Since v1

Files: src/components/membership/QgivJoin.tsx:81 · src/pages/donate.astro:263 · src/components/membership/qgiv.ts:40 | **OWASP:** A08:2021 – Software & Data Integrity Failures

Unchanged in substance since v1; the injection point has moved from `MembershipPage.tsx` to the rebuilt membership flow and now also appears on the donate page. The QGiv embed script is still injected into the DOM with no `integrity` attribute, so a compromise of QGiv's CDN would deliver JavaScript straight into the payment context.

FIX — COMPUTE THE HASH AND ADD THE INTEGRITY ATTRIBUTE

```
# 1. Download the current script and compute its SHA-384 hash
curl -s https://secure.qgiv.com/resources/core/js/embed.js \
  | openssl dgst -sha384 -binary | openssl base64 -A

# 2. Add the hash at both injection points
script.integrity = "sha384-<computed-hash>";
script.crossOrigin = "anonymous";
```

The v6 caveat still applies: SRI pins a version, so confirm with QGiv whether `embed.js` is versioned or updated in place, and coordinate hash updates with their release schedule if it is the latter.

SECURITY POSTURE PROGRESS — ALL ROUNDS

ROUND	DATE	CRITICAL	HIGH	MEDIUM	LOW	TOTAL OPEN	FIXED
v1	2026-03-31	3	5	6	3	17	—
v2	2026-04-02	1	6	7	3	17	3
v3	2026-04-06	0	2	5	3	10	10
v4	2026-04-09	0	1	2	3	6	16
v5	2026-04-13	0	1	2	4	7	17
v6	2026-04-14	0	0	0	1	1	24
v7	2026-09-22	0	1	4	6	11	25

Count note: v6 reported "24 of 25 findings resolved" while its register lists 26 IDs. v7 counts the register — 26 findings from rounds v1–v6, of which 25 are fixed and L-2 remains open — plus the 10 new findings, for 36 total. The v1–v6 rows above are reproduced as those reports stated them.

The jump from 1 open to 11 is a scope effect as much as a quality one: v2–v6 were four-day deltas over a stable codebase, while v7 covers five months in which the site was re-platformed onto SSR, six API routes were added and the entire membership and donation flow was rebuilt. Five of the eleven open items are in code added during the window (S-1, S-3, S-5, S-9, S-10); the other six sit in code or infrastructure configuration that predates it — L-2, tracked since v1, plus S-2, S-4, S-6, S-7 and S-8, which are first reported here after widening the review beyond the recently changed files.

REMEDIATION PLAN — V7

PRIORITY	ACTION	ID	EFFORT
Immediate	Verify the payment server-side before granting a Hub invite or writing to EmailOctopus — restore the signature-verified QGiv webhook, or validate the transaction id against QGiv's API from the endpoint.	S-1	0.5–1 day
This week	Emit `X-Content-Type-Options`, `Referrer-Policy` and `Permissions-Policy` from middleware, and add `frame-ancestors 'none'` to the CSP.	S-2	1 hour
This week	Force `no-store` on `/admin/*` in the cache middleware.	S-3	15 min
This month	Confirm the thirteen 500s from the 300-request burst raised a Sentry alert; if not, fix the alerting gap.	S-4	1 hour
This month	Add Sentry query-string scrubbing; check existing issues for the feed token and rotate it if it appears.	S-5	1–2 hours
This month	Cache the Notion and ICS reads through `resolveWithCache()`, as `/api/projects` already does — the substantive fix for S-4.	S-4	half day
This month	Bump svelte and axios; remove `react-router-dom` and `install`; add `npm audit` to CI.	S-6	1–2 hours
Backlog	Re-derive the GET rate-limit threshold so it sits below the point where the uncached routes 500, and record it with the rule.	S-4	30 min
Backlog	Tighten `/api/pipe`'s origin policy to match the other endpoints.	S-9	15 min
Backlog	Replace `dangerouslySetInnerHTML` in the volunteer form with text, before the proxy is reconfigured.	S-8	15 min
Backlog	Pin SHA-384 hashes for the QGiv embed and the proxied Plausible script.	L-2, S-10	2 hours + vendor
Backlog	Plan the Astro 5 → 7 upgrade; retire the nonce `meta` tag and the prototype patches in favour of an Emotion cache nonce.	S-6, S-7	multi-day

GOOD PRACTICES CONFIRMED IN THE V7 CODEBASE

- **Zero secrets in committed code.** `wrangler.toml` holds only a KV namespace binding; a pattern scan of the whole tracked tree for keys, tokens and Notion secrets returns nothing. `.env.example` lists 20 variables with empty values and a comment explaining each.
- **The nonce-based CSP survived the rewrite intact.** The live header carries `'nonce-…' 'strict-dynamic'` on `script-src` and `'nonce-…' 'self'` on `style-src`, with no `'unsafe-inline'` on either, plus `object-src 'none'` and `base-uri 'self'`. New third parties added since v6 were scoped to the narrowest directive that works — GrowthBook to `connect-src` only, Calendly and YouTube to `frame-src` only.
- **Both new secret comparisons are constant-time.** The admin Basic auth (`basicAuth.ts:27`) and the Sentry webhook HMAC (`sentry-webhook.ts:17`) both go through `constantTimeEqual()`. The webhook verifies a real HMAC-SHA256 over the raw body before parsing it, and rejects any request that carries an `Origin` header at all.
- **project-proxy is still hardened end to end.** Dot-segment normalisation runs before the `/api/method/` prefix guard, the normalised path is what builds the upstream URL, the four-entry `FORWARD_HEADERS` allowlist is unchanged, and `Authorization` is added server-side only.
- **Every API response is non-cacheable, and read-only CORS is correctly scoped.** Verified live: `/api/events` returns `no-store` with `Access-Control-Allow-Origin: *`, while the write endpoints echo a single explicit origin from `corsHeaders()` and 403 anything else.
- **Upstream URL fields are sanitised before they reach the page.** `sanitizeProjectUrls()` runs every ProjectHub URL field through `sanitizeUrl()`, which admits only `http:` and `https:` — and the cache key is versioned (`?v=1`) so tightening those rules invalidates entries stored under the old ones. That is a thoughtful detail.
- **PII redaction held.** Every log and error context in the donation and membership handlers passes `[redacted]@domain` rather than the address, including the new `donation-complete` route.
- **The projects cache is a good model for the rest.** `projectsCachePolicy.ts` is pure, separately testable, refuses to let an empty upstream response overwrite a good entry, and serves stale for 24 hours during an outage. Extending it to the Notion routes would close most of S-4.

METHOD AND SCOPE

Static review of the full diff between `917a0ac` (v6, 2026-04-14) and `7c7a563` (2026-09-21) — 115 commits, 296 files, +43,384 / −5,963 lines — with line-by-line reading of all 15 API routes, the middleware chain, the layouts, the security utilities (`getEnv`, `crypto`, `basicAuth`, `origin`), the data clients, and the client-side components that call privileged endpoints. Supplemented by a dependency advisory review (`npm audit` against the committed lockfile) and the non-destructive live probes logged on page 4. Each of the 25 findings closed in v1–v6 was re-checked against current code.

Not covered: the ProjectHub, Hub and Mattermost services themselves; Cloudflare account and dashboard configuration beyond what the live responses reveal; the Notion workspace permission model; and authenticated behaviour of `/admin/conversions`, which was reviewed as source only since no credentials were used.

Bottom line

The security work from rounds v1–v6 has proved durable: it survived a re-platform and a near-total front-end rewrite with no regressions. What this round shows is a different failure mode — the rules were followed in the code that was reviewed against them, and the gap opened where new architecture moved a decision somewhere the rules did not describe: payment verification moved out of a signed webhook and into the browser. That one finding, S-1, is the whole of the urgent work. Two smaller patterns are worth recording. Controls that live in the Cloudflare dashboard rather than the repository cannot be reviewed alongside the code, and S-4 shows the cost — the GET rate limit is real, but it no longer matches what the last report documented, and nobody would have noticed from the codebase. And S-2, S-6 and S-7 were all reachable at v6 and were not found, because a delta audit only looks at what changed. Both argue for writing down the out-of-repo controls where a code review can see them, and for periodically auditing the whole surface rather than only the diff.